

P2527R0

`std::variant_alternative_index` and `std::variant_alternative_index_v`

Published Proposal, 2021-01-18

This version:

<http://wg21.link/P2527R0>

Author:

[Alex Christensen](#) (Apple)

Audience:

SG18

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

<https://github.com/achristensen07/papers/blob/master/source/P2527r0.bs>

Abstract

A description of `std::variant_alternative_index` and `std::variant_alternative_index_v` which polish use of `std::variant` in C++.

Table of Contents

- 1 Introduction
- 2 Addition to Standard Library
- 3 Ill-formed use
- 4 Motivating Use Case: Migrating C code to use `std::variant`

5 Motivating Use Case: Simple object serialization

§ 1. Introduction

`std::variant` seems have a missing piece. `std::get` and `std::get_if` can be used with an index or with a type. There is already a way to get a type from an index (`std::variant_alternative`) but there is no way to get an index from a type. This adds such a mechanism.

§ 2. Addition to Standard Library

Add the following to the existing header :

```
template<class Type, class Variant> struct variant_alternative_index {  
    static constexpr std::size_t value;  
};  
  
template<class Type, class Variant> inline constexpr  
    std::size_t variant_alternative_index_v = variant_alternative_index<Type, Variant>::value;
```

The design is similar to `std::variant_size` and `std::variant_size_v`. The variadic version is omitted because they are not associated with `std::variant` but rather any list of types. They could be added if feedback indicates their inclusion is desired.

§ 3. Ill-formed use

Like the versions of `std::get` and `std::get_if` that take a type, the program must be ill formed if the type is not a unique element in the types of the `std::variant` such as in these two cases:

```
using Example1 = std::variant<int, double, double>;  
auto ambiguous = std::variant_alternative_index_v<double, Example1>;  
  
using Example2 = std::variant<int, double>;  
auto missing = std::variant_alternative_index_v<float, Example2>;
```

§ 4. Motivating Use Case: Migrating C code to use `std::variant`

Consider the following C program with a unit test:

```
#include <assert.h>

struct NumberStorage {
    enum Type { TYPE_INT, TYPE_DOUBLE } type;
    union {
        int i;
        double d;
    };
};

struct NumberStorage packageInteger(int i) {
    struct NumberStorage packaged;
    packaged.type = TYPE_INT;
    packaged.i = i;
    return packaged;
}

int main() {
    struct NumberStorage i = packageInteger(5);
    assert(i.type == TYPE_INT);
}
```

In order to migrate it from using a `union` to using a `std::variant` one of the cleanest solutions looks something like this:

```
#include <assert.h>
#include <variant>

// Dear future developers: VariantIndex and NumberStorage must stay in sync
// If you reorder or add to one, you must do the same to the other.
enum class VariantIndex : std::size_t { Int, Double };
using NumberStorage = std::variant<int, double>;

NumberStorage packageInteger(int i) {
    return { i };
}

int main() {
```

```

    auto i = packageInteger(5);
    auto expectedIndex = static_cast<std::size_t>(VariantIndex::Int);
    assert(i.index() == expectedIndex);
}

```

Instead, using `std::variant_alternative_index_v` would make the code look cleaner and be easier to maintain:

```

#include <assert.h>
#include <variant>

using NumberStorage = std::variant<int, double>;

NumberStorage packageInteger(int i) {
    return { i };
}

int main() {
    auto i = packageInteger(5);
    auto expectedIndex = std::variant_alternative_index_v<int, NumberStorage>;
    assert(i.index() == expectedIndex);
}

```

§ 5. Motivating Use Case: Simple object serialization

Another place where `std::variant_alternative_index_v` is useful is when we have an index from a source such as deserialization and we want to decide what type it represents without using any magic numbers:

```

struct Reset { };
struct Close { };
struct RunCommand { std::string command; };

using Action = std::variant<Reset, Close, RunCommand>;

void serializeAction(const Action& action, std::vector<uint8_t>& buffer)
{
    buffer.push_back(action.index());
    if (auto* runCommand = std::get_if<RunCommand>(&action))

```

```

        serializeString(runCommand->command, buffer);
    }

    std::optional<Action> deserializeAction(std::span<const uint8_t> source)
    {
        if (!source.size())
            return std::nullopt;

        switch (source[0]) {
            case std::variant_alternative_index_v<Reset, Action>:
                return Reset { };
            case std::variant_alternative_index_v<Close, Action>:
                return Close { };
            case std::variant_alternative_index_v<RunCommand, Action>:
                return RunCommand { deserializeString(source.subspan(1)) };
        }

        return std::nullopt;
    }
}

```

Like the other motivating use case, this could be done with an enum class or `#define RESET_INDEX 0` etc., but this is nicer and references the variant instead of requiring parallel metadata. This was the use case that motivated an implementation in [WebKit](#).